Part C: Big-O Algorithmic Complexity Analysis

Big-O Algorithmic Complexity Analysis Matrix

Operation	Target Code / Function	Best Case	Average Case	Worst Case	Space Complexity	Mathematical Justification
Unamortized Insert	`add_item()`	O(1)	O(1)	O(n)	O(n)	O(1) when capacity remains. Worst case O(n) occurs when array reaches capacity, requiring dynamic reallocation and copying n elements to a new heap buffer.
Linear Deletion	`remove_item_at()`	O(1)	O(n)	O(n)	O(1)	Deleting the last element takes O(1). Deleting at index 0 requires shifting n-1 elements left. The legacy double loop adds constant-factor overhead without changing asymptotic bounds.
Linear Search	`findItem()`	O(1)	O(n)	O(n)	O(1)	Best case O(1) occurs if target is at index 0. Worst case O(n) occurs if target is at index n-1 or absent.
Array Display	`printAll()`	O(n)	O(n)	O(n)	O(1)	Sequentially visits and displays every element from index 0 to n-1.

Operation	Target Code / Function	Best Case	Average Case	Worst Case	Space Complexity	Mathematical Justification
Matrix Transpose	`processMatrix()`	O(R x C)	O(R x C)	O(R x C)	O(R x C)	Requires visiting every cell in an R x C grid via nested iteration loops.

Detailed Breakdown & Mathematical Justifications

1. Unamortized Insert (`add_item()`)

- **Best Case — O(1):** When the current element count is less than `maxCapacity`, inserting an item takes constant time as it simply assigns the value at index `currentCount` and increments the count.
- **Worst Case — O(n):** When `currentCount == maxCapacity`, the array triggers a resize operation. A new array of size `2 * maxCapacity` is allocated on the heap, all `n` existing elements are copied over, and the old buffer is deallocated.
- **Space Complexity — O(n):** The space occupied grows linearly with the number of elements n stored in the dynamic array.

2. Linear Deletion (`remove_item_at()`)

- **Best Case — O(1):** Deleting the very last element in the array (`idx == currentCount - 1`) requires no element shifting—just decrementing `currentCount`.
- **Worst / Average Case — O(n):** Deleting at index 0 requires shifting all remaining $n - 1$ elements one position to the left. On average, $n/2$ shifts are performed.
- **Space Complexity — O(1):** Deletion operates in-place without allocating additional memory buffers.

3. Linear Search (`findItem()`)

- **Best Case — O(1):** The target item is located at the first index (`index 0`).
- **Worst / Average Case — O(n):** The target item is located at the last index or does not exist in the list, requiring a full traversal of all n elements.
- **Space Complexity — O(1):** Operates strictly using pointer/index traversal variables with zero additional memory allocation.

4. Array Display (`printAll()`)

- **Best / Average / Worst Case — O(n):** Printing the contents requires traversing each element from index 0 to $n-1$ sequentially.
- **Space Complexity — O(1):** No temporary arrays or auxiliary dynamic memory are allocated during iteration.

5. Matrix Transpose (`processMatrix()`)

- **Best / Average / Worst Case — O(R x C):** Transposing an $R \times C$ matrix requires visiting every single cell via nested $R \times C$ iteration loops.
- **Space Complexity — O(R x C):** Storing the transposed matrix result requires an auxiliary matrix buffer sized $R \times C$.

By: Jan Martin Fermin & Philip Gotinga